GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	**Course Code**	E2UC302B
Program	B.Tech.(AIML)	**Academic Year**	2026–27
Semester/Batch	III	**Lab Assignment No**	07

		o.	
Set	SETA	**Date**	
FacultyIn-Charge	Mr.ArunendraManiTripathi	**Max.Marks**	10
WeekNo.	6	**Duration**	90minutes

CourseOutcome(s)	CO3: Apply SQL join techniques — INNER, NATURAL and OUTER (LEFT/RIGHT/FULL) joins — to combine and retrieve meaningful information from multiple relational tables.

Skill(s) Addressed	SQL Programming, INNER/NATURAL Joins, OUTER Joins, Salary & Department Analysis

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Execute all SQL queries in MySQL Workbench (or the prescribed SQL client).
- Use the dataset/schema given below and retain the same table and column names.
- Show both the SQL query and the corresponding output for every question.
- Save the SQL script as RollNo_Name_DBMS_Lab07_SETA.sql.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Combine Employee and Department using INNER, NATURAL and OUTER (LEFT/RIGHT/FULL) joins to retrieve matched and unmatched records, and understand how each join type handles rows that have no corresponding match in the other table.

3. DATASET / TABLE SCHEMA

Employee and Department tables, as they stand after Week 5 (re-create only if not already present):

```
-- Employee, as extended through Week 5 (includes Joining_Date) — re-create only if not already
present
CREATE TABLE Employee (
  Emp_ID       INT PRIMARY KEY,
  Emp_Name     VARCHAR(30),
  Department   VARCHAR(20),
  Salary       DECIMAL(10,2),
  Experience   INT,
  Joining_Date DATE
);

INSERT INTO Employee VALUES
(1,'Rahul','IT',55000,3,'2022-03-14'),
(2,'Simran','IT',72000,5,'2021-07-01'),
(3,'Farhan','HR',48000,2,'2023-01-20'),
(4,'Divya','Sales',39000,2,'2020-11-05'),
(5,'Vikram','IT',65000,4,'2022-09-10'),
(6,'Meera','HR',61000,6,'2019-06-18'),
(7,'Aman','Sales',45500,3,'2023-04-02'),
(8,'Anjali','IT',83000,7,'2018-12-30'),
(9,'Rohit','HR',52000,4,'2021-02-14'),
(10,'Priya','Sales',47000,5,'2020-08-22');

-- Department, as created in Week 5 — re-create only if not already present
CREATE TABLE Department (
```

```
  Dept_Name VARCHAR(20) PRIMARY KEY,
  Location  VARCHAR(20),
  HOD       VARCHAR(30)
);

INSERT INTO Department VALUES
('IT','Bengaluru','Anjali Rao'),
('HR','Delhi','Vivek Anand'),
('Sales','Mumbai','Kabir Shah');
```

New this week — enable NATURAL JOIN and add one unmatched row on each side, so every join type has something meaningful to show:

```
-- Rename Department's key column so it matches Employee.Department exactly
-- (required for NATURAL JOIN, which auto-matches identically-named columns)
ALTER TABLE Department RENAME COLUMN Dept_Name TO Department;

-- One new employee whose department does NOT exist in Department (for LEFT/FULL OUTER JOIN)
INSERT INTO Employee VALUES (11,'Zara','Finance',58000,3,'2024-02-01');

-- One new department with NO employees yet (for RIGHT/FULL OUTER JOIN)
INSERT INTO Department VALUES ('Legal','Chennai','Neha Kapoor');
```

4. QUESTIONS / TASKS

Q1. Write an INNER JOIN query (using explicit JOIN ... ON syntax) to combine Employee and Department, displaying Emp_Name, Department, Location and HOD. State how many rows are returned and explain why.

Q2. Extend the query from the previous task with an additional condition to display only employees whose department's Location is 'Bengaluru'.

Q3. Rewrite the INNER JOIN from Q1 using NATURAL JOIN (now possible since Department's key column has been renamed to match Employee.Department). Confirm it returns the same 10 rows.

Q4. Write a LEFT OUTER JOIN to display ALL employees, including 'Zara' (Department = 'Finance'), who has no matching row in Department. The Department-side columns should show NULL for this row.

Q5. Using the LEFT OUTER JOIN from the previous task, add a WHERE condition (checking for NULL on a Department-side column) to isolate exactly which employee(s) have no matching department.

Q6. Write a RIGHT OUTER JOIN to display ALL departments, including 'Legal', which has no matching employee yet. The Employee-side columns should show NULL for this row.

Q7. Using the RIGHT OUTER JOIN from the previous task, add a WHERE condition (checking for NULL on a Employee-side column) to isolate exactly which department(s) have no matching employee.

Q8. MySQL has no native FULL OUTER JOIN keyword. Write a query that emulates one — using UNION of the LEFT OUTER JOIN and RIGHT OUTER JOIN queries above — to display every employee and every department, matched or not. State the total row count.

Q9. Tabulate the row counts returned by INNER, LEFT OUTER, RIGHT OUTER and FULL OUTER JOIN for this dataset (Q1, Q4, Q6, Q8), and explain in 3-4 lines why each count differs.

5. STUDY MATERIAL / REFERENCE

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- JOIN Clause (INNER, LEFT, RIGHT, NATURAL, CROSS): https://dev.mysql.com/doc/refman/8.0/en/join.html
- UNION Clause (used here to emulate FULL OUTER JOIN): https://dev.mysql.com/doc/refman/8.0/en/union.html
- ALTER TABLE ... RENAME COLUMN: https://dev.mysql.com/doc/refman/8.0/en/alter-table.html

6. EVALUATION RUBRIC

Eval	Mar

uationCriteria	ks
CorrectINNERJOINusingexplicitJOIN..ONsyntax	2

CorrectNATURALJOINandcomparisonwithINNERJOIN	1
CorrectLEFTOUTER	2

JOINidentifyingunmatchedrows	
CorrectRIGHTOUTERJOINidentifying	2

unmatchedrows	
CorrectFULLOUTERJOIN(viaUNION)withrow-countveri	2

fication	
Output verification, script quality and viva	1
TOTAL	**10**

7. EVALUATION PROCESS

1. Submission check — SQL script and outputs received within the deadline.
2. Execution check — queries are executed successfully and outputs are verified.
3. Rubric scoring — marks are awarded against the stated criteria.

4. Viva/spot-check — brief questions may be asked to verify individual understanding.

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	**Course Code**	E2UC302B
Program	B.Tech.(AIML)	**Academic Year**	2026–27
Semester/Batch	III	**Lab Assignment No**	08

		o.	
Set	SETB	**Date**	
FacultyIn-Charge	Mr. Arunendra Mani Tripathi	**Max. Marks**	10
WeekNo.	6	**Duration**	90 minutes

CourseOutcome(s)	CO3: Apply SQL join techniques — INNER, NATURAL and OUTER (LEFT/RIGHT/FULL) joins — to combine and retrieve meaningful information from multiple relational tables.

Skill(s) Addressed	SQL Programming, INNER/NATURAL Joins, OUTER Joins, Academic Data Analysis

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Execute all SQL queries in MySQL Workbench (or the prescribed SQL client).
- Use the dataset/schema given below and retain the same table and column names.
- Show both the SQL query and the corresponding output for every question.
- Save the SQL script as RollNo_Name_DBMS_Lab08_SETB.sql.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Combine Student and Department using INNER, NATURAL and OUTER (LEFT/RIGHT/FULL) joins to retrieve matched and unmatched records, and understand how each join type handles rows that have no corresponding match in the other table.

3. DATASET / TABLE SCHEMA

Student and Department tables, as they stand after Week 5 (re-create only if not already present):

```
-- Student, as extended through Week 5 (includes Admission_Date) — re-create only if not already
present
CREATE TABLE Student (
  Student_ID     INT PRIMARY KEY,
  Name           VARCHAR(30),
  Department     VARCHAR(20),
  Semester       INT,
  CGPA           DECIMAL(3,2),
  Attendance     DECIMAL(5,2),
  Admission_Date DATE
);

INSERT INTO Student VALUES
(101,'Rahul','AIML',4,8.40,88.5,'2023-08-01'),
(102,'Anjali','AIML',4,9.10,92.0,'2023-08-01'),
(103,'Amit','CSE',4,7.20,71.0,'2023-08-02'),
(104,'Priya','CSE',4,8.10,80.0,'2023-08-02'),
(105,'Neha','ECE',4,8.80,90.5,'2023-08-03'),
(106,'Rohit','ECE',4,6.90,68.0,'2023-08-03'),
(107,'Simran','AIML',4,8.00,74.5,'2023-08-01'),
(108,'Farhan','CSE',4,9.40,95.0,'2023-08-02'),
(109,'Divya','ECE',4,7.60,82.0,'2023-08-03'),
(110,'Karan','AIML',4,8.90,89.0,'2023-08-01');

-- Department, as created in Week 5 — re-create only if not already present
```

```
CREATE TABLE Department (
  Dept_Name VARCHAR(20) PRIMARY KEY,
  HOD       VARCHAR(30),
  Building  VARCHAR(20)
);

INSERT INTO Department VALUES
('AIML','Dr. Mehta','Block C'),
('CSE','Dr. Rao','Block A'),
('ECE','Dr. Iyer','Block B');
```

New this week — enable NATURAL JOIN and add one unmatched row on each side, so every join type has something meaningful to show:

```
-- Rename Department's key column so it matches Student.Department exactly
-- (required for NATURAL JOIN, which auto-matches identically-named columns)
ALTER TABLE Department RENAME COLUMN Dept_Name TO Department;

-- One new student whose department does NOT exist in Department (for LEFT/FULL OUTER JOIN)
INSERT INTO Student VALUES (111,'Yusuf','Mechanical',4,7.50,78.0,'2023-08-04');

-- One new department with NO students yet (for RIGHT/FULL OUTER JOIN)
INSERT INTO Department VALUES ('IoT','Dr. Verma','Block D');
```

4. QUESTIONS / TASKS

Q1. Write an INNER JOIN query (using explicit JOIN ... ON syntax) to combine Student and Department, displaying Name, Department, HOD and Building. State how many rows are returned and explain why.

Q2. Extend the query from the previous task with an additional condition to display only students whose department's Building is 'Block A'.

Q3. Rewrite the INNER JOIN from Q1 using NATURAL JOIN (now possible since Department's key column has been renamed to match Student.Department). Confirm it returns the same 10 rows.

Q4. Write a LEFT OUTER JOIN to display ALL students, including 'Yusuf' (Department = 'Mechanical'), who has no matching row in Department. The Department-side columns should show NULL for this row.

Q5. Using the LEFT OUTER JOIN from the previous task, add a WHERE condition (checking for NULL on a Department-side column) to isolate exactly which student(s) have no matching department.

Q6. Write a RIGHT OUTER JOIN to display ALL departments, including 'IoT', which has no matching student yet. The Student-side columns should show NULL for this row.

Q7. Using the RIGHT OUTER JOIN from the previous task, add a WHERE condition (checking for NULL on a Student-side column) to isolate exactly which department(s) have no matching student.

Q8. MySQL has no native FULL OUTER JOIN keyword. Write a query that emulates one — using UNION of the LEFT OUTER JOIN and RIGHT OUTER JOIN queries above — to display every student and every department, matched or not. State the total row count.

Q9. Tabulate the row counts returned by INNER, LEFT OUTER, RIGHT OUTER and FULL OUTER JOIN for this dataset (Q1, Q4, Q6, Q8), and explain in 3-4 lines why each count differs.

5. STUDY MATERIAL / REFERENCE

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- JOIN Clause (INNER, LEFT, RIGHT, NATURAL, CROSS): https://dev.mysql.com/doc/refman/8.0/en/join.html
- UNION Clause (used here to emulate FULL OUTER JOIN): https://dev.mysql.com/doc/refman/8.0/en/union.html
- ALTER TABLE ... RENAME COLUMN: https://dev.mysql.com/doc/refman/8.0/en/alter-table.html

6. EVALUATION RUBRIC

Eva	Mar

luationCriteria	ks
CorrectINNERJOINusingexplicitJOIN..ONsyntax	2

CorrectNATURALJOINandcomparisonwithINNERJOIN	1
CorrectLEFTOUTER	2

JOINidentifyingunmatchedrows	
CorrectRIGHTOUTERJOINidentifying	2

unmatchedrows	
CorrectFULLOUTERJOIN(viaUNION)withrow-countveri	2

fication	
Output verification, script quality and viva	1
TOTAL	**10**

7. EVALUATION PROCESS

5. Submission check — SQL script and outputs received within the deadline.
6. Execution check — queries are executed successfully and outputs are verified.
7. Rubric scoring — marks are awarded against the stated criteria.

8. Viva/spot-check — brief questions may be asked to verify individual understanding.

GALGOTIAS UNIVERSITY

SCHOOL OF ARTIFICIAL INTELLIGENCE

Building Future-Ready AI Talent

LABORATORY ASSIGNMENT

Course Name	Database Management System	**Course Code**	E2UC302B
Program	B.Tech.(AIML)	**Academic Year**	2026–27
Semester/Batch	III	**Lab Assignment No**	09

		o.	
Set	SETC	**Date**	
FacultyIn-Charge	Mr.ArunendraManiTripathi	**Max.Marks**	10
WeekNo.	6	**Duration**	90minutes

CourseOutcome(s)	CO3: Apply SQL join techniques — INNER, NATURAL and OUTER (LEFT/RIGHT/FULL) joins — to combine and retrieve meaningful information from multiple relational tables.

Skill(s) Addressed	SQL Programming, INNER/NATURAL Joins, OUTER Joins, Healthcare Data Analysis

1. GENERAL INSTRUCTIONS

- Attempt all questions individually.
- Execute all SQL queries in MySQL Workbench (or the prescribed SQL client).
- Use the dataset/schema given below and retain the same table and column names.
- Show both the SQL query and the corresponding output for every question.
- Save the SQL script as RollNo_Name_DBMS_Lab09_SETC.sql.
- Submit the SQL script and required screenshots through the LMS.
- Plagiarised or copied submissions will be awarded zero.

2. PROBLEM STATEMENT

Combine Patient and Department using INNER, NATURAL and OUTER (LEFT/RIGHT/FULL) joins to retrieve matched and unmatched records, and understand how each join type handles rows that have no corresponding match in the other table.

3. DATASET / TABLE SCHEMA

Patient and Department tables, as they stand after Week 5 (re-create only if not already present):

```
-- Patient, as extended through Week 5 (includes Admission_Date) — re-create only if not already
present
CREATE TABLE Patient (
  Patient_ID     INT PRIMARY KEY,
  Patient_Name   VARCHAR(30),
  Department     VARCHAR(20),
  Age            INT,
  Bill_Amount    DECIMAL(10,2),
  Admission_Date DATE
);

INSERT INTO Patient VALUES
(1,'Rehan','Cardiology',54,85000,'2026-01-10'),
(2,'Zoya','Pulmonology',29,42000,'2026-01-12'),
(3,'Ishaan','Cardiology',62,95000,'2026-01-15'),
(4,'Tanya','General',45,25000,'2026-01-18'),
(5,'Aryan','Cardiology',70,110000,'2026-01-20'),
(6,'Kavya','Pulmonology',38,38000,'2026-01-22'),
(7,'Suresh','General',58,31000,'2026-01-25'),
(8,'Meera','Pulmonology',33,47000,'2026-01-27'),
(9,'Kabir','Cardiology',67,102000,'2026-01-29'),
(10,'Riya','General',49,28000,'2026-01-30');

-- Department, as created in Week 5 (already named to match Patient.Department) — re-create only if
not already present
```

```
CREATE TABLE Department (
  Department  VARCHAR(20) PRIMARY KEY,
  Building    VARCHAR(20),
  Head_Doctor VARCHAR(30)
);

INSERT INTO Department VALUES
('Cardiology','Block D','Dr. Kapoor'),
('Pulmonology','Block E','Dr. Nair'),
('General','Block F','Dr. Sharma');
```

New this week — enable NATURAL JOIN and add one unmatched row on each side, so every join type has something meaningful to show:

```
-- No rename needed this week — Department's key column already matches Patient.Department.

-- One new patient whose department does NOT exist in Department (for LEFT/FULL OUTER JOIN)
INSERT INTO Patient VALUES (11,'Farah','Oncology',41,52000,'2026-02-02');

-- One new department with NO patients yet (for RIGHT/FULL OUTER JOIN)
INSERT INTO Department VALUES ('Neurology','Block G','Dr. Verma');
```

4. QUESTIONS / TASKS

Q1. Write an INNER JOIN query (using explicit JOIN ... ON syntax) to combine Patient and Department, displaying Patient_Name, Department, Building and Head_Doctor. State how many rows are returned and explain why.

Q2. Extend the query from the previous task with an additional condition to display only patients whose department's Building is 'Block D'.

Q3. Rewrite the INNER JOIN from Q1 using NATURAL JOIN (possible directly, since Department's key column already matches Patient.Department). Confirm it returns the same 10 rows.

Q4. Write a LEFT OUTER JOIN to display ALL patients, including 'Farah' (Department = 'Oncology'), who has no matching row in Department. The Department-side columns should show NULL for this row.

Q5. Using the LEFT OUTER JOIN from the previous task, add a WHERE condition (checking for NULL on a Department-side column) to isolate exactly which patient(s) have no matching department.

Q6. Write a RIGHT OUTER JOIN to display ALL departments, including 'Neurology', which has no matching patient yet. The Patient-side columns should show NULL for this row.

Q7. Using the RIGHT OUTER JOIN from the previous task, add a WHERE condition (checking for NULL on a Patient-side column) to isolate exactly which department(s) have no matching patient.

Q8. MySQL has no native FULL OUTER JOIN keyword. Write a query that emulates one — using UNION of the LEFT OUTER JOIN and RIGHT OUTER JOIN queries above — to display every patient and every department, matched or not. State the total row count.

Q9. Tabulate the row counts returned by INNER, LEFT OUTER, RIGHT OUTER and FULL OUTER JOIN for this dataset (Q1, Q4, Q6, Q8), and explain in 3-4 lines why each count differs.

5. STUDY MATERIAL / REFERENCE

- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- JOIN Clause (INNER, LEFT, RIGHT, NATURAL, CROSS): https://dev.mysql.com/doc/refman/8.0/en/join.html
- UNION Clause (used here to emulate FULL OUTER JOIN): https://dev.mysql.com/doc/refman/8.0/en/union.html
- ALTER TABLE ... RENAME COLUMN: https://dev.mysql.com/doc/refman/8.0/en/alter-table.html

6. EVALUATION RUBRIC

Evalu	Marks

ationCriteria	
CorrectINNERJOINusingexplicitJOIN..ONsyntax	2
Co	1

rrectNATURALJOINandcomparisonwithINNERJOIN	
CorrectLEFTOUTERJO	2

INidentifyingunmatchedrows	
CorrectRIGHTOUTERJOINidentifyingun	2

matchedrows	
CorrectFULLOUTERJOIN(viaUNION)withrow-countverifi	2

cation	
Output verification, script quality and viva	1
TOTAL	**10**

7. EVALUATION PROCESS

9. Submission check — SQL script and outputs received within the deadline.
10. Execution check — queries are executed successfully and outputs are verified.
11. Rubric scoring — marks are awarded against the stated criteria.
12. Viva/spot-check — brief questions may be asked to verify individual understanding.

QUESTION-WISE MAPPING & SOLUTION — SET A

A. Question-wise Mapping

Question	BTL	SOLO Level	CO
Q1	L3 – Apply	Multistructural	CO3
Q2	L3 – Apply	Relational	CO3
Q3	L3 – Apply	Relational	CO3
Q4	L3 – Apply	Relation	CO3

		al	
Q5	L4–Analyze	Relational	CO3
Q6	L3–Apply	Relational	CO3
Q7	L4–Analyze	Relational	CO3
Q8	L4–Analyze	Relational	CO3
Q9	L4–Analyze	Relational	CO3

B. Solution

Execute the following MySQL queries using the Employee / Department dataset in the assignment.

Q1.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E
JOIN Department D ON E.Department = D.Department;
-- Returns 10 rows: only Employee rows with a matching Department are kept.
```

Q2.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E
JOIN Department D ON E.Department = D.Department
WHERE D.Location = 'Bengaluru';
```

Q3.

```
SELECT Emp_Name, Department, Location, HOD
FROM Employee
NATURAL JOIN Department;
-- Same 10 rows as Q1 — NATURAL JOIN auto-matches the identically-named Department column.
```

Q4.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E
LEFT JOIN Department D ON E.Department = D.Department;
-- Returns 11 rows: 10 matched + 1 unmatched (Zara, with NULLs for Location/HOD).
```

Q5.

```
SELECT E.Emp_Name, E.Department
FROM Employee E
LEFT JOIN Department D ON E.Department = D.Department
WHERE D.Department IS NULL;
-- Returns exactly 1 row: Zara (Department = 'Finance').
```

Q6.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E
RIGHT JOIN Department D ON E.Department = D.Department;
-- Returns 11 rows: 10 matched + 1 unmatched ('Legal', with NULLs for Emp_Name/Department).
```

Q7.

```
SELECT D.Department, D.Location, D.HOD
FROM Employee E
RIGHT JOIN Department D ON E.Department = D.Department
WHERE E.Emp_ID IS NULL;
-- Returns exactly 1 row: 'Legal'.
```

Q8.

```
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E LEFT JOIN Department D ON E.Department = D.Department
UNION
SELECT E.Emp_Name, E.Department, D.Location, D.HOD
FROM Employee E RIGHT JOIN Department D ON E.Department = D.Department;
-- Returns 12 rows: 10 matched + 1 unmatched employee + 1 unmatched department. UNION removes the
duplicated matched rows.
```

Q9.

```
-- INNER JOIN      : 10 rows  (only matched pairs)
-- LEFT OUTER JOIN : 11 rows  (matched pairs + unmatched employee rows)
-- RIGHT OUTER JOIN: 11 rows  (matched pairs + unmatched department rows)
-- FULL OUTER JOIN : 12 rows  (matched pairs + BOTH sides' unmatched rows)
-- Explanation: INNER JOIN keeps only rows with a match on both sides. Each OUTER JOIN additionally
-- preserves the unmatched rows from the side(s) it names (LEFT keeps unmatched Employee, RIGHT keeps
-- unmatched Department, FULL keeps both) filling the missing side's columns with NULL.
```

Verification: *Confirm INNER = 10 rows, LEFT OUTER = 11 rows, RIGHT OUTER = 11 rows and FULL OUTER (via UNION) = 12 rows against the given dataset — these counts should match exactly, since only one row was added on each side to create an unmatched case.*

QUESTION-WISE MAPPING & SOLUTION — SET B

A. Question-wise Mapping

Question	BTL	SOLO Level	CO
Q1	L3 – Apply	Multistructural	CO3
Q2	L3 – Apply	Relational	CO3
Q3	L3 – Apply	Relational	CO3
Q4	L3 – Apply	Relation	CO3

		al	
Q5	L4–Analyze	Relational	CO3
Q6	L3–Apply	Relational	CO3
Q7	L4–Analyze	Relational	CO3
Q8	L4–Analyze	Relational	CO3
Q9	L4–Analyze	Relational	CO3

B. Solution

Execute the following MySQL queries using the Student / Department dataset in the assignment.

Q1.

```
SELECT S.Name, S.Department, D.HOD, D.Building
FROM Student S
JOIN Department D ON S.Department = D.Department;
-- Returns 10 rows: only Student rows with a matching Department are kept.
```

Q2.

```
SELECT S.Name, S.Department, D.HOD, D.Building
FROM Student S
JOIN Department D ON S.Department = D.Department
WHERE D.Building = 'Block A';
```

Q3.

```
SELECT Name, Department, HOD, Building
FROM Student
NATURAL JOIN Department;
-- Same 10 rows as Q1 — NATURAL JOIN auto-matches the identically-named Department column.
```

Q4.

```
SELECT S.Name, S.Department, D.HOD, D.Building
FROM Student S
LEFT JOIN Department D ON S.Department = D.Department;
-- Returns 11 rows: 10 matched + 1 unmatched (Yusuf, with NULLs for HOD/Building).
```

Q5.

```
SELECT S.Name, S.Department
FROM Student S
LEFT JOIN Department D ON S.Department = D.Department
WHERE D.Department IS NULL;
-- Returns exactly 1 row: Yusuf (Department = 'Mechanical').
```

Q6.

```
SELECT S.Name, S.Department, D.HOD, D.Building
FROM Student S
RIGHT JOIN Department D ON S.Department = D.Department;
-- Returns 11 rows: 10 matched + 1 unmatched ('IoT', with NULLs for Name/Department).
```

Q7.

```
SELECT D.Department, D.HOD, D.Building
FROM Student S
RIGHT JOIN Department D ON S.Department = D.Department
WHERE S.Student_ID IS NULL;
-- Returns exactly 1 row: 'IoT'.
```

Q8.

```
SELECT S.Name, S.Department, D.HOD, D.Building
FROM Student S LEFT JOIN Department D ON S.Department = D.Department
UNION
SELECT S.Name, S.Department, D.HOD, D.Building
FROM Student S RIGHT JOIN Department D ON S.Department = D.Department;
-- Returns 12 rows: 10 matched + 1 unmatched student + 1 unmatched department. UNION removes the
duplicated matched rows.
```

Q9.

```
-- INNER JOIN      : 10 rows  (only matched pairs)
-- LEFT OUTER JOIN : 11 rows  (matched pairs + unmatched student rows)
-- RIGHT OUTER JOIN: 11 rows  (matched pairs + unmatched department rows)
-- FULL OUTER JOIN : 12 rows  (matched pairs + BOTH sides' unmatched rows)
-- Explanation: INNER JOIN keeps only rows with a match on both sides. Each OUTER JOIN additionally
-- preserves the unmatched rows from the side(s) it names (LEFT keeps unmatched Student, RIGHT keeps
-- unmatched Department, FULL keeps both) filling the missing side's columns with NULL.
```

Verification: *Confirm INNER = 10 rows, LEFT OUTER = 11 rows, RIGHT OUTER = 11 rows and FULL OUTER (via UNION) = 12 rows against the given dataset — these counts should match exactly, since only one row was added on each side to create an unmatched case.*

QUESTION-WISE MAPPING & SOLUTION — SET C

A. Question-wise Mapping

Question	BTL	SOLO Level	CO
Q1	L3 – Apply	Multistructural	CO3
Q2	L3 – Apply	Relational	CO3
Q3	L3 – Apply	Relational	CO3
Q4	L3 – Apply	Relation	CO3

		al	
Q5	L4–Analyze	Relational	CO3
Q6	L3–Apply	Relational	CO3
Q7	L4–Analyze	Relational	CO3
Q8	L4–Analyze	Relational	CO3
Q9	L4–Analyze	Relational	CO3

B. Solution

Execute the following MySQL queries using the Patient / Department dataset in the assignment.

Q1.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P
JOIN Department D ON P.Department = D.Department;
-- Returns 10 rows: only Patient rows with a matching Department are kept.
```

Q2.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P
JOIN Department D ON P.Department = D.Department
WHERE D.Building = 'Block D';
```

Q3.

```
SELECT Patient_Name, Department, Building, Head_Doctor
FROM Patient
NATURAL JOIN Department;
-- Same 10 rows as Q1 — NATURAL JOIN auto-matches the identically-named Department column.
```

Q4.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P
LEFT JOIN Department D ON P.Department = D.Department;
-- Returns 11 rows: 10 matched + 1 unmatched (Farah, with NULLs for Building/Head_Doctor).
```

Q5.

```
SELECT P.Patient_Name, P.Department
FROM Patient P
LEFT JOIN Department D ON P.Department = D.Department
WHERE D.Department IS NULL;
-- Returns exactly 1 row: Farah (Department = 'Oncology').
```

Q6.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P
RIGHT JOIN Department D ON P.Department = D.Department;
-- Returns 11 rows: 10 matched + 1 unmatched ('Neurology', with NULLs for Patient_Name/Department).
```

Q7.

```
SELECT D.Department, D.Building, D.Head_Doctor
FROM Patient P
RIGHT JOIN Department D ON P.Department = D.Department
WHERE P.Patient_ID IS NULL;
-- Returns exactly 1 row: 'Neurology'.
```

Q8.

```
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P LEFT JOIN Department D ON P.Department = D.Department
UNION
SELECT P.Patient_Name, P.Department, D.Building, D.Head_Doctor
FROM Patient P RIGHT JOIN Department D ON P.Department = D.Department;
-- Returns 12 rows: 10 matched + 1 unmatched patient + 1 unmatched department. UNION removes the
duplicated matched rows.
```

Q9.

```
-- INNER JOIN      : 10 rows  (only matched pairs)
-- LEFT OUTER JOIN : 11 rows  (matched pairs + unmatched patient rows)
-- RIGHT OUTER JOIN: 11 rows  (matched pairs + unmatched department rows)
-- FULL OUTER JOIN : 12 rows  (matched pairs + BOTH sides' unmatched rows)
-- Explanation: INNER JOIN keeps only rows with a match on both sides. Each OUTER JOIN additionally
-- preserves the unmatched rows from the side(s) it names (LEFT keeps unmatched Patient, RIGHT keeps
-- unmatched Department, FULL keeps both) filling the missing side's columns with NULL.
```

Verification: *Confirm INNER = 10 rows, LEFT OUTER = 11 rows, RIGHT OUTER = 11 rows and FULL OUTER (via UNION) = 12 rows against the given dataset — these counts should match exactly, since only one row was added on each side to create an unmatched case.*